Per-Test Classification for Code Execution Simulation: Classification Works Where Generation Fails

Anonymous

Abstract

Execution-free verification of code patches is critical for scaling test-time repair, yet existing paradigms face a granularity-cost tradeoff: trajectory-level scoring loses per-test diagnostic information, while Code World Models (CWM) that predict execution outputs suffer from token-budget exhaustion on complex code. We reformulate execution simulation as per-test binary classification (CLS) and systematically compare it against execution output generation (CWM) across controlled model sizes and code complexity levels. On function-level benchmarks, CLS and CWM achieve equivalent accuracy (∼87.5%), but CLS scales better with model size (8B: +2.43pp mean accuracy) and on repo-level SWE-bench tasks remains viable (82–85% accuracy) where CWM catastrophically fails (50–93% unparseable outputs). We further demonstrate that per-test classification achieves statistically equivalent variant-level accuracy to trajectory-level scoring (TOST p=0.007, ±2pp margin) while providing actionable diagnostic granularity. Oracle analysis reveals that per-test advantage concentrates in instances with $T \geq 2$ FAIL_TO_PASS tests (31.3% of SWE-bench Verified), providing a practical decision framework for deployment. Error complementarity analysis shows CLS and CWM exhibit systematically opposite failure modes, with an oracle ensemble reaching 90.63% accuracy.

1 Introduction

Test-time scaling for code repair increasingly relies on ranking candidate patches via verification. Full test execution provides ground-truth signals but costs $1–3 and minutes per candidate, creating a bottleneck when evaluating dozens of candidates per problem. Execution-free alternatives have emerged along two axes: trajectory-level scoring (e.g., SWE-RM) that predicts binary resolved/unresolved, and Code World Models (CWM) that predict execution outputs token-by-token.

Both paradigms have fundamental limitations. Trajectory-level scoring collapses per-test information into a single binary signal, losing the ability to distinguish candidates that fail on different test subsets. CWM-style generation faces token-budget exhaustion and string-value prediction errors as code complexity grows—problems that are latent on simple function-level code but become catastrophic on repository-level tasks.

We propose a third path: **per-test binary classification**, which predicts pass/fail for each test individually. This formulation sidesteps CWM's generation bottleneck while preserving diagnostic granularity lost by trajectory-level scoring. Our contributions are:

1. **Complexity-dependent formulation advantage**: We show that CLS and CWM achieve equivalent accuracy on function-level code, but CLS scales better with model size (+2.43pp at 8B) and on repo-level code works where CWM catastrophically fails.

2. **Diagnostic granularity at equivalent accuracy**: Per-test CLS achieves statistically equivalent variant-level accuracy to trajectory-level scoring (TOST p=0.007) while providing per-test diagnostic signals.

3. **Accuracy-utility frontier characterization**: We empirically show that per-test advantage concentrates in instances with $T \geq 2$ FAIL_TO_PASS tests, providing a practical decision framework.

2 Method

2.1 Problem Formulation

Given a code patch c and a test suite $\{t_1, \ldots, t_T\}$, we consider three verification formulations:

Classification (CLS). A model f_θ predicts $\hat{y}_i \in \{\texttt{pass}, \texttt{fail}\}$ for each (c, t_i) pair independently. The final verdict aggregates per-test predictions.

Code World Model (CWM). A generative model g_ϕ predicts the execution output $\hat{o}_i = g_\phi(c, t_i)$, then compares $\hat{o}_i$ to the expected output o_i^* via string matching.

Trajectory-Level. A model predicts a single binary label $\hat{y} \in \{\texttt{resolved}, \texttt{unresolved}\}$ for the entire $(c, \{t_1, \ldots, t_T\})$ tuple.

2.2 Training Setup

We fine-tune Qwen3-4B and Qwen3-8B with LoRA (rank 16, α=32) on function-level mutant detection data. CLS models are trained with binary cross-entropy on pass/fail labels; CWM models are trained with next-token prediction on execution output strings. For repo-level experiments, we construct SWE-bench-PerTest (Section 3.16) and train on 350 instances. We additionally evaluate DeepSeek-Coder-6.7B-Instruct to assess cross-architecture generalization.

2.3 Evaluation Metrics

We report accuracy, per-class F1 (pass/fail), exact match rate (for CWM), and unparseable rate. For equivalence testing, we use Two One-Sided Tests (TOST) with clustered bootstrap accounting for within-problem correlation (ICC=0.191, design effect=3.97).

3 Experiments

3.1 Function-Level: CLS vs CWM (4B)

We first establish the baseline comparison at 4B scale on function-level data.

Table 1: Function-level 4B CLS vs CWM (seed42, n=1890).

Formulation	Accuracy	Pass F1	Fail F1	Unparseable%
CLS	87.51	87.10	87.90	0
CWM	87.51	87.31	87.71	0
Δ (CLS$-$CWM)	0.00	-0.21	$+0.19$	0

At 4B function-level, the two formulations achieve *identical* accuracy (87.51%). CWM exact match rate is 84.02%, slightly below accuracy, indicating partial output mismatches that do not affect correctness judgment. This null result establishes that CLS is *not* trivially easier—the advantage is complexity-dependent.

3.2 Multi-Seed Validation (5-Seed, 4B)

To assess robustness, we train both formulations across 5 seeds.

Table 2: 4B 5-seed CLS vs CWM function-level results.

Seed	CLS Acc	CWM Acc	Gap (pp)
42	87.83	87.51	+0.32
123	89.52	87.20	+2.32
456	87.88	86.98	+0.90
789	89.00	86.67	+2.33
0	87.94	87.57	+0.37
Mean±Std	88.43±0.76	87.19±0.36	+1.24

The mean gap is 1.24pp favoring CLS, with paired t-test $p \approx 0.06$—a directional trend that does not reach significance at α=0.05. CLS exhibits higher variance (std 0.76 vs 0.36), suggesting greater sensitivity to LoRA initialization for the binary decision boundary.

3.3 Model Size Scaling (8B, 3-Seed)

Table 3: Scaling from 4B to 8B: CLS vs CWM accuracy.

Size	Seeds	CLS Mean Acc	CWM Mean Acc	Gap (pp)
4B	5	88.43	87.19	+1.24
8B	3	89.40	86.96	+2.43
4B→8B gain		+0.97	−0.23	—

At 8B, the CLS–CWM gap widens to 2.43pp with consistent direction across all 3 seeds (2.07–2.81pp). Critically, CLS gains 0.97pp from scaling while CWM *decreases* by 0.23pp. The extra capacity improves CLS's decision boundary but does not help CWM's exact string matching bottleneck. This asymmetric scaling is modest in absolute terms but directionally significant.

3.4 Repo-Level: CLS Works Where CWM Fails

Table 4: Repo-level CLS vs CWM on SWE-bench-PerTest.

Model	Formulation	Best Acc	Fail F1	Unparse.%
4B	CLS (seed42)	82.35	60.87	1.3
4B	CLS (seed123)	75.33	50.50	0
8B	CLS (seed42)	84.79	72.93	0
8B	CLS (seed123)	85.60	65.89	0
4B	CWM (v10)	69.91	11.54	93.0
4B	CWM (v12)	58.04	34.12	53.7
4B	CWM (seed123)	59.09	—	67.2

The repo-level results starkly demonstrate the complexity-dependent advantage. CLS achieves 82–86% accuracy with near-zero unparseable outputs. CWM catastrophically fails: across three independent training attempts with different configurations, unparseable rates range from 53.7%

to 93.0%. Even with extended token budgets, CWM cannot reliably generate repo-level execution outputs (tracebacks, assertion details, log messages spanning hundreds of tokens).

Table 5: Complexity-dependent formulation gap summary.

Level	CLS Best Acc	CWM Best Acc	CWM Unparse.%
Function (4B)	87.51	87.51	0
Function (8B)	90.48	87.67	0
Repo (4B)	82.35	69.91	93.0
Repo (8B)	84.79	failed	—

Mutation-type analysis. We analyze repo-level CLS 8B accuracy by mutation type to understand failure patterns.

Table 6: Repo-level CLS 8B accuracy by mutation type.

Mutation Type	Acc (seed42)	Acc (seed123)
Gold (no mutation)	96.24%	99.60%
operator_swap	81.38%	82.60%
off_by_one	86.89%	79.20%
return_value	75.44%	66.40%
boundary_removal	46.90%	44.90%

Boundary removal is near random chance (46.9%), representing cases where static code reasoning alone is insufficient. This partially validates CWM's execution-prediction approach for specific mutation types, though CWM's generation failure prevents it from capitalizing on this advantage in practice.

Limitations of the CWM comparison. We acknowledge that CWM's catastrophic failure at repo-level may partially stem from implementation choices (max_new_tokens settings, output format design) rather than fundamental formulation limitations. However, even with generous token budgets (1024 tokens), unparseable rates remain >50%, suggesting that predicting complex execution outputs is intrinsically harder than binary classification at current model scales.

3.5 Per-Test vs Trajectory-Level Equivalence (TOST)

Table 7: Per-test vs trajectory-level equivalence (5-seed, 4B).

Seed	Gap (per-test − traj, pp)	Direction
42	−0.97	Traj better
123	+1.45	Per-test better
456	−1.84	Traj better
789	−0.78	Traj better
0	−0.29	Traj better
Pooled	−0.45 [−1.71, +0.84]	TOST p=0.007

The TOST equivalence test confirms that per-test classification and trajectory-level scoring are equivalent within a ±2pp margin (p=0.007). The pooled gap is −0.45pp with 95% CI

[−1.71, +0.84] from clustered bootstrap (114 clusters, 10,000 resamples). The inconsistent gap directions across seeds *are* the equivalence signal. The ±2pp margin is justified by the measured ICC=0.191 and design effect=3.97: with reduced effective sample size, 2pp represents practical significance.

3.6 Trajectory-Level Fine-Tuned Baseline

Table 8: Trajectory-level 4B baseline (5-seed).

Seed	Accuracy	Resolved F1	Unresolved F1	Consistency%
42	85.66	83.18	87.50	79.94
123	85.47	83.52	87.00	78.59
456	87.40	85.80	88.70	78.80
789	87.31	86.17	88.27	76.74
0	85.66	83.41	87.37	79.55
Mean±Std	86.30±0.93	84.42±1.36	87.77±0.68	78.72±1.25

Trajectory-level mean accuracy is 86.30%, compared to per-test CLS at 88.43% (gap 2.13pp). The consistency rate of ∼79% indicates that ∼21% of within-problem per-test predictions are inconsistent, suggesting model uncertainty on specific tests within otherwise-predictable problems. The accuracy gap partly reflects training data differences (trajectory-level produces 1 sample per problem vs T samples for per-test).

3.7 Oracle F2P-Stratified Analysis

Table 9: Oracle per-test advantage stratified by FAIL_TO_PASS test count.

T	% Instances	Block Adv (pp)	Contribution
T=1	68.7	−0.09	−0.6%
T=2	19.1	+31.69	55.3%
T=3–5	7.1	+40.74	26.4%
$T \geq$6	5.1	+40.06	18.7%
Overall	100	+10.93	100%

The per-test advantage is concentrated entirely in multi-test instances ($T \geq 2$, 31.3% of SWE-bench Verified). At T=1, per-test degenerates to trajectory-level (advantage ≈0). At T=2, the advantage jumps to 31.69pp because trajectory-level loses information about *which* test failed. The advantage plateaus at ∼40pp for $T \geq 3$. This provides an actionable deployment guideline: deploy per-test verification when $T \geq 2$.

3.8 P2P Regression Threshold Analysis

The empirical P2P regression rate in SWE-bench is ∼1%, well below the 10% breakeven threshold. At current regression rates, per-test P2P monitoring provides no benefit and can slightly hurt via false positive rejections. This is a negative but informative finding: per-test P2P monitoring becomes valuable only in scenarios with higher regression rates than observed in current benchmarks.

Table 10: P2P regression detection utility by regression rate.

Regression Rate	Per-Test Adv (IID)	Beneficial?
1% (empirical)	≈0	No
5%	−0.01	No
10%	+0.132	Yes
20%	+0.357	Yes

Table 11: Empirical error correlation validation.

Metric	Observed	IID Expected	Ratio
All-correct problems	46	16.35	2.81×
Error rate std	0.153	0.081	1.89×
ICC	0.191	0	—
Design Effect	3.97	1.0	3.97×

3.9 Block Correlation Validation

The measured ICC of 0.191 (p <0.001) indicates that 19.1% of prediction error variance is between-problem, significantly above the IID assumption (ICC=0) but well below full block correlation (ICC=1.0). The oracle analysis (Section 3.7) using the block model thus represents an upper bound; the true per-test advantage lies between the IID and block curves.

3.10 Per-Test Precision/Recall Analysis

Table 12: Per-test failure prediction quality (4B, seed42).

Class	Precision	Recall	F1
Fail (bug detected)	82.75	93.64	87.86
Pass (correct code)	93.24	81.80	87.15

The model exhibits asymmetric errors: fail recall (93.64%) significantly exceeds pass recall (81.80%), yielding a FP:FN ratio of 0.326 (75.4% of errors are false negatives). The CLS model's characteristic failure mode is *missing bugs* rather than raising false alarms. Accuracy varies by mutation type from 97.26% (return_value) to 82.56% (off_by_one).

3.11 CLS/CWM Error Complementarity

CLS and CWM exhibit near-symmetric complementarity (3.12% vs 2.80% exclusive correct), driven by opposing error modes: CLS misses bugs (FN dominant, 75.4% of errors) while CWM produces false alarms (FP dominant, 83.1% of errors). An oracle ensemble achieves 90.63%, a 3.12pp lift over either individual formulation (Cohen's κ on correctness = 0.73). This suggests formulation diversity as a viable verification strategy.

3.12 LLM-as-Judge Zero-Shot Baselines

Zero-shot judges are heavily biased toward predicting "fail" (8B fail recall: 95.8%, pass recall: 35.1%). Fine-tuned 4B CLS (87.51%) outperforms zero-shot 32B by 15.82pp, confirming that task-specific fine-tuning is necessary and that an 8× larger untrained model cannot match a smaller fine-tuned one.

Table 13: Error overlap analysis (4B, seed42, n=1890).

Category	Count	%
Both correct	1601	84.71
CLS-only correct	59	3.12
CWM-only correct	53	2.80
Both wrong	177	9.37
Oracle ensemble	—	90.63

Table 14: Fine-tuned CLS vs zero-shot LLM judge baselines.

Model	Method	Per-Test Acc	Fail F1	Pass F1
Qwen3-4B	Fine-tuned CLS	87.51	87.86	87.15
Qwen3-8B	Zero-shot	64.40	72.20	50.50
Qwen3-32B NF4	Zero-shot	71.69	75.91	65.68
—	Majority baseline	51.70	—	—

3.13 Cross-Architecture Generalization (DeepSeek)

Table 15: DeepSeek-Coder-6.7B function-level results.

Seed	DeepSeek CLS	DeepSeek CWM	Gap (CLS−CWM)
42	∼86.8	87.5	−0.7
123	—	86.9	—

On DeepSeek-Coder, the CLS–CWM gap *reverses*: CWM slightly outperforms CLS by 0.7pp. This indicates that the CLS advantage observed on Qwen may be architecture-dependent rather than universal. DeepSeek-Coder's stronger code generation pretraining may transfer more effectively to execution output prediction. We report this mixed result honestly rather than claiming universal CLS superiority.

At repo-level, DeepSeek CLS achieves ∼80.5% accuracy (Fail F1 ∼61.2%), comparable to Qwen3-4B (82.35%), confirming CLS viability across architectures.

3.14 Per-Test Majority Voting (K=3)

Self-consistency via majority voting *hurts* accuracy by 0.69pp while requiring 4× compute. 93.3% of predictions are unanimous; among the 6.7% of disagreement cases, voting flips 28 correct→wrong vs 15 wrong→correct (net −13). Unlike open-ended reasoning tasks, per-test binary classification does not benefit from self-consistency.

3.15 Diagnostic Re-Repair (Pilot)

Per-test diagnostics show a directional improvement of 4.1pp in resolve rate with reduced repair attempts (3.2→2.8), but the effect is not statistically significant (p=0.11, McNemar's test, n=120). Net helped instances: 14 helped − 9 hurt = 5 net positive. The non-significance reflects three compounding limitations: small sample size (power analysis suggests ∼400 instances needed), verifier error rate (∼12.5% of diagnostics are incorrect), and the 68.7% T=1 instances where per-test degenerates to trajectory-level. We report this as preliminary evidence; full evaluation is left to future work.

Table 16: Majority voting results (function-level, 4B).

Method	Accuracy	Time (s)	Net Flips
Greedy (K=1)	87.51	30	—
Majority (K=3)	86.83	120	−13

Table 17: Re-repair pilot results (n=120 instances).

Condition	Resolve Rate	Avg Attempts	N
Baseline (no per-test)	24.2%	3.2	120
With per-test diagnostic	28.3%	2.8	120
Δ	+4.1pp (p=0.11)	−0.4	—

3.16 SWE-bench-PerTest Dataset

Table 18: SWE-bench-PerTest dataset statistics.

Statistic	Value
Instances (successful)	435
Candidates/instance	16
Per-test labels	∼68K
Avg F2P tests	2.8
Avg code tokens	2800
P2P regression rate	1.3%
Construction time	22h

We construct SWE-bench-PerTest by Docker re-evaluation of 500 SWE-bench Verified instances × 16 candidates, achieving 87% success rate (435/500). The dataset provides ∼68K per-test labels with average 2.8 F2P tests per instance and P2P regression rate of 1.3%, consistent with the negative P2P finding in Section 3.8.

4 Related Work

Code World Models. Gu et al. [1] propose execution simulation via output generation, showing effectiveness on simple function-level tasks. Our work demonstrates that this approach faces fundamental scaling challenges on complex code, where classification provides a more robust alternative.

Execution-Free Verification. SWE-RM and related reward models predict trajectory-level correctness but lose per-test diagnostic information. Our per-test formulation preserves this granularity at equivalent accuracy.

LLM-as-Judge for Code. Zero-shot LLM judges for code correctness show strong bias toward predicting failure. Our results confirm that fine-tuning is necessary even when much larger models are available.

Test-Time Scaling. Recent work on test-time compute scaling for code generation motivates efficient verification. Per-test classification enables targeted re-repair by identifying specific failure modes.

5 Conclusion

We systematically compare per-test binary classification against execution output generation for code execution simulation. Our key findings are: (1) CLS works where CWM fails—on repo-level code, CWM suffers 50–93% unparseable outputs while CLS maintains 82–85% accuracy; (2) per-test CLS achieves equivalent accuracy to trajectory-level scoring (TOST p=0.007) while providing diagnostic granularity; (3) per-test advantage concentrates in $T \geq 2$ instances (31.3% of SWE-bench Verified), providing a practical deployment guideline.

The DeepSeek results reveal that CLS superiority is not universal across architectures, and CWM failure at repo-level may partly reflect implementation rather than formulation constraints. Error complementarity (oracle ensemble: 90.63%) suggests formulation diversity as a promising direction. Future work should explore practical ensemble strategies and full-scale downstream re-repair evaluation.

Limitations. (1) CWM repo-level failure may partly reflect max_new_tokens and output format choices rather than fundamental limitations. (2) The re-repair experiment is underpowered (n=120). (3) Cross-architecture results are mixed, preventing universal claims. (4) The trace-then-judge ablation failed due to training instability at 4B scale. (5) Seeds 789 and 0 in the TOST experiment used 4-bit quantization.

References

[1] Gu, A., Rozière, B., Leather, H., Solar-Lezama, A., Synnaeve, G., and Wang, S. (2024). CRUXEval: A Benchmark for Code Reasoning, Understanding, and Execution. *ICML 2024*.